实验2.1 传感数据解析

学号	姓名	专业年级	课程老师
21080012078	姚嫣如	网络空间安全23	赵硕

一、实验目的

根据 GM10 传感设备，理解 GM10 传感设备的通信协议，通过传感设备的通信协议解析出返回的数据，掌握**字符型通信协议**解析过程，为后续的传感设备数据采集软件编程打下基础。

二、实验要求

预习 GM10 传感设备的通信协议，通过实验，熟练掌握根据 GM10 传感设备 的**通信协议解析数据的方法**。

三、实验内容

根据提供的 GM10 传感设备通信协议，对文件中的 GM10 返回数据进行解析和显示。

GM10 通信协议数据格式:

```
EA<crlf>      // 数据起始符

DATE_yy/mo/dd <crlf>     // 数据日期

TIME_hh:mm:ss.mmmt<crlf>  // 数据时间

s_cccca1a2a3a4uuuuuuuuuufddddddddE-pp <crlf>

…

s_cccca1a2a3a4uuuuuuuuuufddddddddE-pp<crlf>

EN<crlf>      // 数据结束符
```

- t预留(空白)
- s数据状态（N标准、D差值输入、S不使用、O溢出、E错误、B断偶）
- cccc通道号，即传感器在GM10种的编号

- a1a2a3a4报警状态（H上限报警、L下限报警、h差值上限报警、空白无报警）
- uuuuuuuuuu单位(固定10字符。左对齐输出，不足10字符的部分输出空白。)
- 科学计数法E（指数为10的倍数）：

  f符号(+或-)

  ddddddddd尾数部分(00000000～99999999，8位)

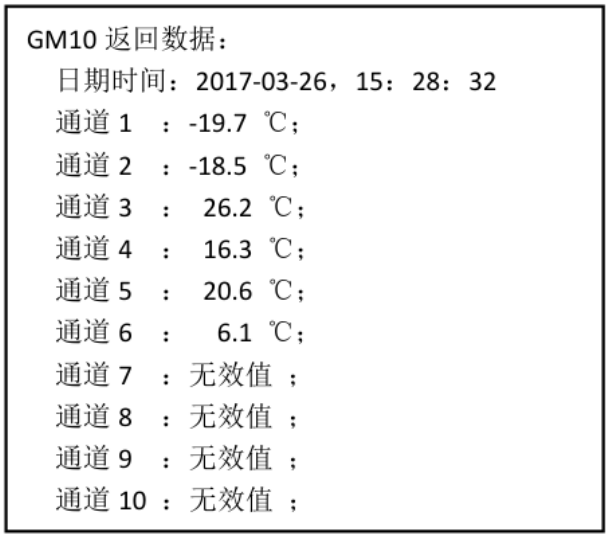
  pp指数部分(00～05)

  👉 +00000239E-1 = 23.9
- 每一行以回车+换行作为结束符。

四、实验步骤

1. 学习 GM10 传感设备的通信协议。
2. 打开仿真器桌面上的“琏雾实验系统\实验2.1”文件夹，利用 **C#语言** 编写程序，读取gm10data.txt文件中的数据，根据 GM10 通信协议对文件中的数据进行解析和显示。

   👉 GM10解析数据显示示例如图所示：



五、实验结果

（一）编程调试

1. 下载 C#

   终端成功下载截图如下：

```
C:\Users\yrr>dotnet -h
使用情况: dotnet [runtime-options] [path-to-application] [arguments]

执行 .NET 应用程序。

runtime-options:
  --additionalprobingpath <path>   要探测的包含探测策略和程序集的路径。
  --additional-deps <path>         指向其他 deps.json 文件的路径。
  --depsfile                       指向 <application>.deps.json 文件的路径。
  --fx-version <version>           要用于运行应用程序的安装版共享框架的版本。
  --roll-forward <setting>         前滚至框架版本(LatestPatch, Minor, LatestMinor, Major, LatestMajor, Disable)。
  --runtimeconfig                  指向 <application>.runtimeconfig.json 文件的路径。

path-to-application:
  要执行的应用程序 .dll 文件的路径。

使用情况: dotnet [sdk-options] [command] [command-options] [arguments]

执行 .NET SDK 命令。

sdk-options:
  -d|--diagnostics  启用诊断输出。
  -h|--help         显示命令行帮助。
  --info            显示 .NET 信息。
  --list-runtimes   显示安装的运行时。
  --list-sdks       显示安装的 SDK。
  --version         显示使用中的 .NET SDK 版本。
```

2. 配置VS code

下载扩展并生成项目:

- `dotnet new console` 生成新项目
- `dotnet run` 运行代码

```
PS D:\yyr\study\物联网\实验2.1> dotnet new console
已成功创建模板“控制台应用”。

正在处理创建后操作...
正在还原 D:\yyr\study\物联网\实验2.1\实验2.1.csproj:
已成功还原。


PS D:\yyr\study\物联网\实验2.1> dotnet run
GM10返回数据:
日期时间: 2022-01-04, 04:50:40
通道1: -19.7℃;
通道2: -19.5℃;
```

3. 编写代码

完整代码如下:

```
using System;

using System.IO;

using System.Linq;

namespace GM10DataAnalysis

{
```

```
class Program
{
 static void Main(string[] args)
 {
  // 1. 读取文件并过滤空行
  string filePath = @"D:\yyr\study\物联网\实验2.1\gm10data.txt";
  if (!File.Exists(filePath))
  {
   Console.WriteLine("无该文件，解析失败！");
   Console.ReadKey();
   return;
  }
  string[] allLines = File.ReadAllLines(filePath)
       .Select(line => line.TrimEnd('\r', '\n')) // 移除换行符
       .Where(line => !string.IsNullOrWhiteSpace(line))
       .ToArray();
  string dateStr = string.Empty;
  string timeStr = string.Empty;
  Console.WriteLine("GM10返回数据：");
  // 2. 解析日期和时间
  foreach (var line in allLines)
  {
   if (line.StartsWith("DATE"))
   {
    // DATE_yy/mo/dd 格式：截取空格后的部分
    string datePart = line.Split(new[] { ' ' },
StringSplitOptions.RemoveEmptyEntries)[1];
    string[] dateParts = datePart.Split('/');
    dateStr = $"20{dateParts[0]}-{dateParts[1]}-{dateParts[2]}";
   }
   else if (line.StartsWith("TIME"))
```

```
    {
     // TIME_hh:mm:ss.mmmt 格式：截取空格后的部分，去除毫秒和预留位
     string timePart = line.Split(new[] { ' ' },
StringSplitOptions.RemoveEmptyEntries)[1];
     timeStr = timePart.Split('.')[0]; // 保留hh:mm:ss
    }
   }
   Console.WriteLine($"日期时间：{dateStr}，{timeStr}");
   // 3. 解析传感数据行（固定位置截取，核心修正）
   foreach (var line in allLines)
   {
    // 仅处理传感数据行（以状态字符开头：N/D/S/O/E/B）
    if (line.Length >= 20 && "NDSOEB".Contains(line[0].ToString()))
    {
     try
     {
      // 固定位置截取所有字段（关键：不依赖空格分割）
      string dataStatus = line[0].ToString();       // 数据状态（s）
      string channelNoStr = line.Substring(2, 4).TrimStart('0'); // 通道号
（cccc），去前导零
      string alarmStatus = line.Substring(6, 4).Trim(); // 报警状态
（a1a2a3a4），去空格
      string unit = line.Substring(10, 10).Trim();     // 单位（10字符），去空
格
      string valuePart = line.Substring(20); // 数值部分（fddddddddE-pp）
      // 解析数值部分
      double actualValue = 0;
      bool isInvalid = false;
      if (valuePart.Length >= 12) // 数值部分至少12位（+00000000E-00）
      {
       char sign = valuePart[0]; // 符号（+/-）
```

```
      string mantissa = valuePart.Substring(1, 8); // 8位尾数
      int exponentIndex = valuePart.LastIndexOf("E-") + 2; // 指数部分前的E位置
      string exponentPart = valuePart.Substring(exponentIndex, 2); // 指数pp
      // 数值转换（异常处理：避免格式错误导致崩溃）
      if (double.TryParse(mantissa, out double mantissaNum) && int.TryParse(exponentPart, out int exponent))
      {
       actualValue = (sign == '+' ? 1 : -1) * mantissaNum * Math.Pow(10, -exponent);
       actualValue = Math.Round(actualValue, 1); // 保留1位小数
      }
      else
      {
       isInvalid = true;
      }
     }
     else
     {
      isInvalid = true;
     }
     // 判断数据是否有效（结合状态和解析结果）
     string valueDesc = string.Empty;
     if (isInvalid || dataStatus != "N")
     {
      // 非N状态（D/S/O/E/B）或数值解析失败，判定为无效值
      valueDesc = "无效值";
      // 补充状态说明
      switch (dataStatus)
      {
```

```
                case "D": valueDesc += "（差值输入）"; break;
                case "S": valueDesc += "（不使用）"; break;
                case "O": valueDesc += "（溢出）"; break;
                case "E": valueDesc += "（错误）"; break;
                case "B": valueDesc += "（断偶）"; break;
            }
        }
        else
        {
            valueDesc = ${actualValue:F1};
            // 补充报警状态（如有）
            if (!string.IsNullOrEmpty(alarmStatus))
            {
                valueDesc += $"（报警：{alarmStatus}）";
            }
        }
        // 输出结果（含报警状态和有效性判断）
        Console.WriteLine($"通道{channelNoStr}：{valueDesc}{unit}；");
    }
    catch (Exception ex)
    {
        // 单个通道解析失败不影响整体，输出错误提示
        Console.WriteLine($"通道解析失败：{line} | 错误：{ex.Message}");
    }
  }
  return; // 结束解析
 }
}
```

4. 运行结果

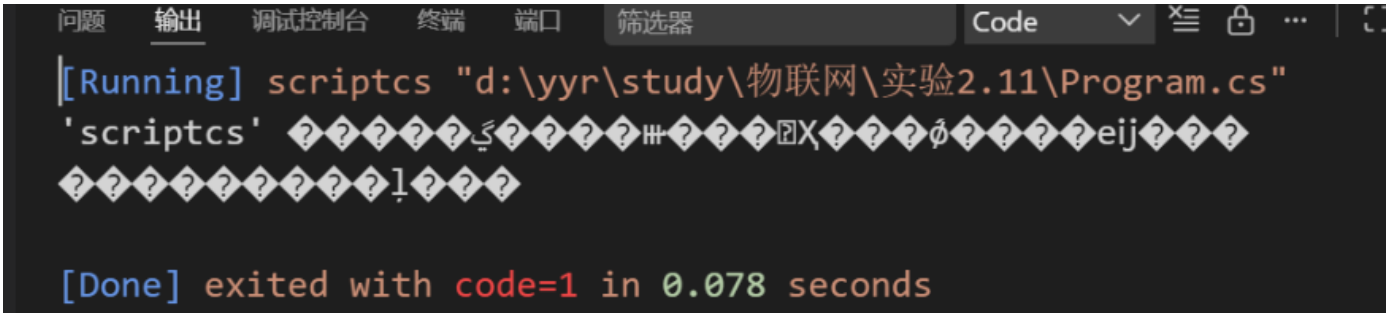
运行结果如下：

共910个通道数据。

```
PS D:\yyr\study\物联网\实验2.1> dotnet run
GM10返回数据:
日期时间: 2022-01-04, 04:50:40
通道1: -19.7℃;
通道2: -19.5℃;
通道3: -20.2℃;
通道4: 24.8℃;
通道5: -20.3℃;
通道6: 4.2℃;
通道7: 3.3℃;
通道8: 3.2℃;
通道9: -1.9℃;
通道10: -0.3℃;
通道101: -0.4℃;
通道102: 0.7℃;
通道103: 0.6℃;
通道104: -22.5℃;
通道105: -20.8℃;
通道106: -19.1℃;
通道107: -21.2℃;
通道108: -19.4℃;
通道109: -17.8℃;
通道110: -23.9℃;
通道201: -23.8℃;
通道202: -22.7℃;
通道203: -19.2℃;
通道204: -21.4℃;
通道205: -20.0℃;
通道206: -21.1℃;
通道207: -0.3℃;
通道208: -0.4℃;
通道209: 0.7℃;
通道210: 0.6℃;
通道301: -21.5℃;
通道302: -22.2℃;
通道303: -21.9℃;
通道304: -21.3℃;
```

5. 调试问题及解决方法

初次运行，只运行了Program.cs单个文件，导致运行结果出现报错



解决办法：**.NET 控制台项目必须通过 `dotnet run` 命令整体运行**，不能直接“运行单个 CS 文件”。

六、实验心得

通过本次 GM10 采集器数据解析实验，我对字符型通信协议的解析流程有了清晰的认识，收获如下：

在**协议认知层面**，我理解了 GM10 数据采集器的通信协议规范，掌握了从EA起始符到EN结束符的完整数据帧结构，明确了状态位、通道号、报警状态、符号位、尾数和指数等字段的位置与含义，认识到工业通信协议对数据格式的严格要求。

在**实践编程层面**，我学会了用 C# 读取文本文件并按协议逐位解析数据，解决了数据行截取、科学计数法转换、异常值判断等问题，体会到“按协议定位字段”是解析字符型数据的核心方法，也认识到代码鲁棒性在工业场景中的重要性。

在**系统认知层面**，我完成了从原始协议数据到可读温度数据的转换，实现了实验要求的格式化输出，理解了“传感设备→采集器→计算机解析”的完整数据流转过程，为后续物联网数据采集软件开发打下了基础。

本次实验让我深刻体会到，严谨的协议理解和规范的代码实现是工业数据可靠解析的关键，也提升了我处理实际工程问题的能力。